Rapport de stage

Étude et optimisation de l'algorithme DPLL pour la résolution du problème SAT

Tuteur entreprise : Yu Li & Corinne Lucet

Tuteur universitaire : Yu Li

2025

Oscar Wintrebert | **LABORATOIRE MIS**

Tables des matières :

1. Présentation de l'organisme d'accueil **2**
2. Présentation de la mission confiée **2**
Objectifs précis de mon stage : 3
3. Travail réalisé **4**
3.1 Implémentation initiale de DPLL sans propagation unitaire 5
3.2 Optimisation par propagation unitaire 6
3.3 Ajout d'une heuristique de choix de variable (Jeroslow-Wang) 7
3.4 Tests, validation et analyse comparative des versions 8
4. Bilan d'apprentissage **9**
5. Conclusion et perspectives **10**

1. Présentation de l'organisme d'accueil

Mon stage s'est déroulé au sein du laboratoire MIS (Modélisation, Information & Systèmes) de l'Université de Picardie Jules Verne (UPJV), situé sur le campus d'Amiens. Le MIS est un laboratoire pluridisciplinaire spécialisé dans les Sciences et Techniques de l'Information et de la Communication (STIC). Il regroupe plusieurs équipes de recherche aux thématiques variées.

J'ai été intégré à l'équipe OCIA (Optimisation, Cryptographie & Intelligence Artificielle), qui travaille sur des problématiques liées à l'intelligence artificielle et à la résolution de problèmes complexes. Les chercheurs de cette équipe développent notamment des algorithmes avancés permettant de résoudre efficacement des problèmes combinatoires difficiles comme le problème SAT, ainsi que des problématiques liées à la cryptographie et à l'apprentissage automatique.

Cette diversité thématique a créé un environnement propice à la réalisation de mon stage, me permettant à la fois d'appliquer mes connaissances en algorithmique et programmation, et de découvrir des approches modernes mêlant méthodes classiques et intelligence artificielle.

2. Présentation de la mission confiée

Mon stage, effectué du 7 avril au 31 mai 2025 dans l'équipe OCIA du laboratoire MIS, s'inscrit dans un projet de recherche plus large visant à étudier les capacités des modèles de langage de type LLM (Large Language Models) à résoudre des problèmes complexes, tels que le SAT ou le p-centre. Dans ce contexte global, ma mission spécifique a consisté en une étude approfondie du problème SAT et de ses méthodes classiques de résolution, afin de mieux comprendre comment ces méthodes pourraient interagir ou se comparer aux approches par réseaux de neurones récents (notamment les Transformers).

Le problème SAT (SATisfiability problem ou problème de satisfaisabilité booléenne) est l'un des problèmes les plus importants en informatique théorique. Il consiste à déterminer s'il existe une assignation des variables booléennes d'une formule logique qui rende cette formule vraie. Autrement dit : existe-t-il une façon d'attribuer à chaque variable la valeur vrai ou faux, de telle sorte que l'ensemble de la formule devienne vrai ?

En pratique, le problème SAT est généralement représenté sous la forme normale conjonctive (CNF). Cette forme consiste en une conjonction (ET logique) de plusieurs clauses, chaque clause étant elle-même une disjonction (OU logique) d'un ou plusieurs

littéraux. Un littéral représente une variable ou sa négation. Ainsi, une formule SAT typique en CNF peut se présenter ainsi :

$(X1 \vee \neg X2 \vee X3) \wedge (\neg X1 \vee X2) \wedge (X2 \vee X3 \vee \neg X4)$

Dans cette représentation :

Chaque clause entre parenthèses est une disjonction de littéraux.
La formule complète est une conjonction de ces clauses.

Ce problème possède une importance historique particulière : il a été le premier problème reconnu NP-complet, prouvé par Stephen Cook en 1971 dans le célèbre théorème de Cook-Levin. Cette propriété implique que, sauf avancée majeure en complexité algorithmique, il n'existe pas actuellement d'algorithme polynomial connu capable de résoudre efficacement toutes les instances possibles du problème SAT.

Cependant, malgré cette complexité théorique élevée, le problème SAT a une importance pratique majeure et apparaît dans de nombreux domaines d'application en informatique. Parmi ces applications concrètes, on peut citer :

- La vérification formelle de logiciels et matériels informatiques : s'assurer qu'un programme informatique ou une puce électronique se comporte correctement en toutes circonstances.

- La résolution de problèmes d'optimisation combinatoire : par exemple, trouver une combinaison optimale d'éléments sous contraintes spécifiques, comme le problème du voyageur de commerce ou les affectations optimales de ressources.

- Les problèmes de planification automatique : établir des plans d'action efficaces dans des contextes complexes comme la gestion logistique, les plannings d'usine ou les stratégies militaires.

- Les jeux logiques et puzzles, tels que le Sudoku, les grilles logiques, ou encore certains casse-têtes où les contraintes peuvent être formulées logiquement.

Dans ce cadre, disposer d'algorithmes efficaces capables de résoudre rapidement des formules SAT complexes demeure un enjeu important tant sur le plan scientifique que sur le plan industriel.

Objectifs précis de mon stage :

Pour aborder ce problème SAT dans le contexte du projet de recherche global, il m'a été confié les objectifs suivants :

1. Étudier en profondeur le problème SAT (ses différentes représentations en CNF, ses formulations particulières) et les méthodes classiques de résolution, notamment l'algorithme de DPLL (Davis–Putnam–Logemann–Loveland).

2. Implémenter en langage C un solveur DPLL de base, puis l'optimiser en y intégrant des améliorations reconnues comme la propagation unitaire.

3. Tester et comparer les performances des différentes versions développées sur un ensemble d'instances réelles tirées de la base SATLib, une bibliothèque d'instances SAT reconnue dans la communauté de recherche.

4. Enfin, me familiariser avec les approches modernes à base de Transformers appliquées à SAT, afin de situer mon travail dans un contexte innovant plus large. Il s'agissait de comprendre en particulier comment un Transformer pourrait être entraîné pour résoudre des formules SAT ou comment le problème SAT pourrait être formulé pour être abordé efficacement par ces modèles récents. Pour cela, j'ai notamment consulté les recherches et publications récentes sur le sujet.

En résumé, ma mission combinait une composante algorithmique et développement (implémenter un solveur SAT performant en langage C) à une composante recherche bibliographique (explorer les approches modernes de résolution basées sur les réseaux de neurones, notamment les Transformers). Ce stage représentait donc une synthèse riche entre l'informatique théorique (logique, algorithmique, complexité) et l'informatique appliquée (programmation bas-niveau en C, analyse expérimentale de performances), avec en toile de fond une comparaison instructive entre les méthodes de résolution classiques et les approches modernes d'intelligence artificielle.

3. Travail réalisé

Pour atteindre les objectifs, le travail a suivi un déroulement en plusieurs étapes, correspondant généralement aux améliorations successives du solveur DPLL ainsi qu'aux tâches d'analyse associées. Je détaille ci-dessous les étapes principales du développement, la structure du programme réalisé, ainsi que les résultats obtenus et l'analyse des performances de chaque version.

3.1 Implémentation initiale de DPLL sans propagation unitaire

Dans un premier temps, j'ai implémenté une version basique de l'algorithme DPLL en langage C, sans aucune optimisation particulière. Cette version correspond à l'algorithme de backtracking classique pour SAT : on vérifie d'abord si la formule est triviale (clause vide ou formule vide), puis on choisit une variable non assignée et on explore récursivement les deux possibilités (assigner la variable à Vrai, ou à Faux). Si aucune branche ne mène à une solution satisfaisante, l'algorithme retourne *insatisfiable* (faux). Cette version a été développée dans un fichier `main2.c` et se contente de parcourir exhaustivement l'arbre de recherche des assignations possibles. Structure de l'algorithme DPLL basique : en pseudo-code, l'algorithme effectue à chaque appel récursif :

1. Test de terminaison – cas satisfaisant : si toutes les clauses de la formule sont satisfaites par l'affectation courante, alors une solution a été trouvée et l'algorithme renvoie vrai (SAT).
2. Test de conflit immédiat : si une clause est devenue vide (c'est-à-dire qu'aucun littéral de cette clause ne peut être vrai sous l'affectation courante), alors l'affectation courante mène à un conflit et l'algorithme renvoie faux, déclenchant un retour en arrière.
3. Choix d'une variable et récursion : sinon, on sélectionne une variable non encore assignée (dans cette version initiale, le choix est fait de manière simple, par exemple la prochaine variable dans l'ordre des indices, sans heuristique particulière). On assigne cette variable à *vrai* et on appelle récursivement DPLL. Si cette branche renvoie vrai (solution trouvée), on propage le résultat. Sinon, on assigne la variable à *faux* et on appelle récursivement. Si aucune des deux branches ne rend la formule satisfaisable, on retourne faux pour remonter au niveau supérieur de récursion.

Dans cette première implémentation, aucune simplification de la formule n'est effectuée en dehors du backtracking lui-même. Concrètement, cela signifie que même si un littéral pourrait être déduit directement (unitaire) ou si une clause est satisfaisée par une affectation partielle, le programme continue d'explorer toutes les combinaisons, y compris des branches inutiles. Cela a un impact négatif notable sur les performances, car l'arbre de recherche exploré est bien plus grand qu'il ne pourrait l'être avec des optimisations. Après avoir programmé cette version en C, j'ai également réalisé une petite implémentation de DPLL en Java dans le but de l'intégrer à un solveur de Sudoku (projet annexe). Cette étape, bien que non centrale, m'a aidé à vérifier la compréhension de l'algorithme sur un cas concret (SAT est facilement applicable aux Sudoku en encodant les règles du jeu sous forme logique). Elle a confirmé le fonctionnement correct de l'algorithme et servi

d'introduction à la lecture des fichiers CNF au format DIMACS (utilisé pour représenter des instances SAT : une première ligne `p cnf nbVar nbClauses` puis une liste de clauses).

3.2 Optimisation par propagation unitaire

La deuxième étape majeure a été d'optimiser le solveur DPLL en y intégrant la propagation unitaire (PU). La propagation unitaire est une optimisation essentielle des solveurs SAT : à chaque étape, si une clause ne contient plus qu'un seul littéral non assigné (appelée *clause unitaire*), alors ce littéral doit nécessairement être vrai pour que la formule puisse être satisfaisable (sinon cette clause serait insatisfaisable). On peut donc assigner d'office cette valeur et simplifier la formule en conséquence, sans avoir à explorer le cas contraire qui mène forcément à un conflit.

On applique récursivement cette règle tant que des clauses unitaires apparaissent. Cette optimisation permet de réduire drastiquement l'espace de recherche en éliminant tôt des branches inutiles. J'ai implémenté la propagation unitaire dans une nouvelle version du programme (`main.c`). Concrètement, après les tests de clauses satisfaites ou vides, j'ajoute une boucle qui détecte les clauses unitaires et assigne la variable correspondante, en simplifiant la formule (on marque les clauses satisfaites par cette affectation, on raccourcit les autres clauses en enlevant le littéral faux assigné, etc.). Une fois toutes les simplifications faites, l'algorithme poursuit le déroulement normal (choix d'une variable restante et appels récursifs).

Structure mémoire et gestion de la formule : pour représenter la formule CNF en C, j'ai choisi une structure basée sur des tableaux dynamiques de clauses. Chaque clause est un tableau dynamique de littéraux (entiers positifs ou négatifs représentant les variables ou leurs négations). J'ai mis en place des fonctions de lecture du fichier DIMACS, de construction de ces structures, ainsi que de redimensionnement lorsqu'une clause est modifiée (par exemple, lors de la propagation unitaire, on réduit la taille des clauses auxquelles un littéral faux a été enlevé). C'est lors de cette phase que j'ai rencontré des problèmes de gestion mémoire : un bug notable a été un segmentation fault causé par un redimensionnement insuffisant d'un tableau de littéraux, que j'ai dû corriger en ajustant l'allocation mémoire allouée aux clauses. J'ai profité de cette version optimisée pour améliorer l'affichage et le suivi de l'algorithme. J'ai ajouté un affichage de l'arbre de décision DPLL de façon indentée, ainsi qu'un chronomètre (via la bibliothèque `<time.h>`, en utilisant `clock()` pour mesurer le temps CPU consommé). L'affichage de l'arbre permet de visualiser les appels récursifs et les décisions prises, ce qui a été utile sur de petites instances pour vérifier le bon déroulement de la propagation unitaire (on voit que dès qu'une unité est trouvée, elle est propagée sans branches alternatives, ce qui raccourcit l'arbre). Naturellement, cet affichage détaillé a été désactivé pour les tests de performance sur de grandes instances, car il ralentit inutilement le programme.

3.3 Ajout d'une heuristique de choix de variable (Jeroslow-Wang)

En parallèle de l'optimisation par propagation unitaire, j'ai étudié puis intégré une heuristique permettant de guider le choix des variables à chaque étape du backtracking, dans le but de rendre l'algorithme DPLL plus efficace. L'heuristique que j'ai choisie est celle de Jeroslow-Wang, connue pour ses bons résultats dans les solveurs SAT classiques.

Le principe de cette heuristique est de privilégier les variables qui apparaissent fréquemment dans les clauses, en donnant plus de poids aux occurrences dans les clauses les plus courtes. En effet, une variable présente dans de nombreuses clauses courtes a plus de chances d'avoir un impact fort sur la suite de la résolution. L'objectif est donc d'orienter la recherche vers les parties « critiques » de la formule, en espérant soit trouver rapidement une solution, soit détecter un conflit plus tôt.

Le score attribué à une variable (v) est donné par la formule : $J(\text{v}) = \sum_{C \ni v} 2^{-|C|}$

où $|C|$ représente la taille (le nombre de littéraux) de la clause C, et où la somme porte sur toutes les clauses contenant v ou $\neg v$. En d'autres termes, plus une variable apparaît dans des clauses courtes, plus son score est élevé. L'algorithme choisit ensuite la variable non assignée ayant le score le plus élevé pour effectuer le branchement.

L'intégration de cette heuristique dans le programme main3 (version enrichie de main.c) s'est traduite par l'ajout d'une fonction de calcul des scores pour toutes les variables non encore assignées, à chaque étape où une décision doit être prise. Cette fonctionnalité a été ajoutée après les versions initiales du solveur (sans optimisation, puis avec propagation unitaire), afin de pouvoir mesurer l'apport de cette heuristique en complément des optimisations précédentes.

L'impact sur le code a été significatif : pour appliquer l'heuristique, il a fallu parcourir toutes les clauses de la formule afin de compter les occurrences et calculer les scores associés. Cela représente un certain coût en temps de calcul, en particulier sur les formules contenant de nombreuses clauses. Pour limiter cet impact, j'ai mis en place un recalcul partiel des scores, en ne mettant à jour que les scores des variables affectées par la dernière décision (c'est-à-dire après simplification de la formule suite à la propagation unitaire). Cette optimisation reste cependant basique, et une implémentation plus avancée pourrait s'appuyer sur des structures de données spécifiques pour gérer les scores de manière plus efficace, notamment en les mettant à jour de façon incrémentale.

Faute de temps, j'ai privilégié une solution simple et fonctionnelle, qui s'est révélée suffisante pour traiter les instances de taille modérée testées durant le stage.

3.4 Tests, validation et analyse comparative des versions

Une fois les trois versions du solveur DPLL finalisées (sans propagation unitaire, avec propagation unitaire, et avec propagation unitaire + heuristique Jeroslow-Wang), j'ai mené une série de tests à l'aide d'instances standard de la base SATLib : uf20-91 (20 variables), uf50-218 (50 variables) et uf75-325 (75 variables), connues pour leur difficulté.

Instance SATLib	Clauses	Variables	Temps sans PU en (ms)	Temps avec PU en (ms)	Temps avec PU et JW en (ms)	Facteur d'accélération
uf20-91	91	20	2,86	0,4	x	6,6858
uf50-218	218	50	112 566	15	3,8	6682 x 7,8
uf75-325	325	75	x	108	11,6	18,14

Figure 1 : Tableau des temps moyens d'exécution des différentes versions de DPLL (instances SATLib)

Les résultats montrent clairement l'impact de chaque optimisation.

Propagation unitaire :
Sur les petites instances (uf20-91), l'ajout de la propagation unitaire a permis des accélérations typiques de 6 à 10 fois.
Par exemple, uf20-01.cnf passe de 4,9 ms à 0,5 ms.
Sur les instances à 50 variables, le gain est encore plus spectaculaire :
uf50-01.cnf passe de 172 secondes à 26 ms, soit un facteur > 6600.

Heuristique Jeroslow-Wang :
L'ajout de l'heuristique améliore encore les temps.
Sur uf50-01.cnf, le temps chute à 1,6 ms, soit 16 fois plus rapide que PU seul.
Sur certaines grandes instances (uf75-325), l'amélioration peut atteindre jusqu'à 100 fois selon les cas.

Résumé :

- ➢ La version 1 (sans PU) est vite dépassée en performance.
- ➢ La version 2 (avec PU) apporte un gain essentiel.
- ➢ La version 3 (PU + heuristique) est la plus efficace, surtout sur les grandes formules, avec un surcoût négligeable au regard des bénéfices.

Ces tests ont permis d'illustrer concrètement l'apport de chaque optimisation classique du DPLL, tout en montrant les limites d'une implémentation simple face à des solveurs industriels.

4.6 Comparaison avec le projet Hélios

Le projet Hélios, développé en parallèle dans le cadre du même groupe de recherche, explore une approche par apprentissage automatique pour la résolution du problème SAT, à l'aide d'un modèle de type Transformer. L'objectif est de tester la capacité de ces modèles à prédire, étape par étape, une séquence de décisions menant à une affectation satisfaisante des variables d'une formule CNF.

J'ai pu consulter le code du modèle Hélios, qui génère des prédictions successives (par exemple des affectations ou des choix de littéraux), mais son fonctionnement étant très différent de celui d'un solveur logique comme DPLL, une comparaison directe avec mon programme n'a pas pu être mise en place durant la période du stage. De plus, l'absence de métriques communes (temps de résolution, taux de succès sur un corpus, etc.) rendait difficile une évaluation rigoureuse.

Cependant, cette cohabitation de deux approches l'une logique et déterministe, l'autre statistique et prédictive ouvre des perspectives intéressantes pour des travaux futurs. Une piste possible serait d'utiliser les prédictions d'un modèle comme Hélios pour guider les décisions d'un solveur DPLL classique, ou inversement, d'entraîner un modèle à partir des parcours d'un solveur exact. Ce type d'approche hybride, combinant les forces de l'intelligence artificielle et de l'algorithmique classique, représente aujourd'hui un enjeu actif de recherche.

4. Bilan d'apprentissage

Ce stage de deux mois a été très enrichissant et m'a permis de mobiliser et d'acquérir de nombreuses compétences, tout en identifiant des axes de progression pour la suite de mon parcours.

1. Compétences mobilisées : J'ai puisé dans plusieurs acquis de ma formation. Tout d'abord, mes connaissances en logique formelle et combinatoire ont été sollicitées pour comprendre le problème SAT et manipuler les formules booléennes (CNF, clauses, littéraux). Les compétences en algorithmique et récursivité ont été cruciales pour implémenter et structurer l'algorithme DPLL (qui est récursif par nature). La maîtrise du langage C et de la gestion mémoire s'est avérée indispensable pour développer un programme fiable : j'ai dû manipuler des pointeurs, faire des allocations dynamiques (avec `malloc/realloc`) et assurer l'absence de fuites de mémoire (en libérant correctement toutes les structures en fin d'exécution). Enfin, des notions de lecture/écriture de fichiers ont été mises en œuvre pour traiter le format DIMACS et gérer les entrées/sorties du solveur.

2. Ce stage m'a permis d'apprendre énormément. J'ai découvert comment analyser les performances d'un programme en mesurant les temps d'exécution et en identifiant les parties les plus lentes, comme la propagation unitaire. J'ai aussi approfondi mes connaissances sur les heuristiques dans les algorithmes de recherche, en particulier celle de Jeroslow-Wang. J'ai appris à mener des tests de manière rigoureuse, à organiser les résultats dans des tableaux et à les interpréter pour comprendre ce qui fonctionne ou non. Ce savoir-faire me sera utile pour de futurs projets techniques ou scientifiques. Enfin, je me suis initié aux modèles de langage et aux Transformers, et j'ai compris comment ces approches peuvent être utilisées pour résoudre des problèmes logiques, ce qui a élargi ma culture en intelligence artificielle.

3. Ce stage m'a aussi montré des domaines où j'ai encore des progrès à faire. En programmation bas niveau, j'ai vu que la gestion efficace de la mémoire en C est complexe, et qu'il existe des techniques avancées pour améliorer les performances que je ne connais pas encore. Du côté de l'intelligence artificielle, mes connaissances en apprentissage automatique sont encore limitées. Je souhaite mieux comprendre le fonctionnement des modèles récents comme les Transformers, aussi bien en théorie qu'en pratique, avec des outils comme PyTorch. Enfin, j'aimerais découvrir comment fonctionnent les solveurs SAT professionnels les plus avancés, afin de mieux situer mon travail et m'inspirer de leurs techniques.

5. Conclusion et perspectives

Ce stage a été une étape importante dans mon parcours. Il m'a permis de relier des notions abordées en cours, comme la logique propositionnelle ou la résolution de Sudoku par transformation en SAT, à leur mise en œuvre concrète dans un solveur que j'ai codé moi-même en C. J'ai pu comprendre en détail le fonctionnement de l'algorithme DPLL, et mesurer l'impact réel d'optimisations classiques comme la propagation unitaire ou les heuristiques de choix de variable.

Sur le plan technique, j'ai renforcé mes compétences en programmation dans un langage bas niveau, dans un contexte où la performance et la robustesse du code comptent. J'ai aussi acquis une méthode de travail rigoureuse : construire des tests, analyser les résultats, identifier les limites d'une implémentation et améliorer le programme étape par étape.

Ce stage m'a également permis d'élargir ma culture en informatique, notamment en découvrant comment des approches modernes, comme les modèles de langage de type Transformer, peuvent s'appliquer à des tâches logiques. Cela m'a donné envie d'en apprendre davantage sur l'interaction entre IA et algorithmes classiques.

À l'avenir, je souhaiterais approfondir ces domaines, que ce soit dans un Master en informatique ou à travers d'autres projets. Ce stage m'a confirmé mon intérêt pour les problèmes combinatoires et les techniques de résolution, qu'elles soient exactes ou apprenantes, et m'a donné de solides bases pour progresser dans cette voie.